

[중간고사 대비]

범위	복습 내용
2장 자료형	숫자, 문자열, 리스트, 튜플, 딕셔너리, 집합, 불, 변수
3장 제어문	if, while, for
4장 입출력	함수, 사용자 입출력, 파일 읽고 쓰기, 프로그램의 입출력
5장 클래스	클래스와 객체, self, 초기화, 상속, 오버라이딩, 클래스 변수

1. 파이썬 기본

질문	설명
10과 "10"의 차이는?	
리스트에서 첫 번째 원소의 인덱스는?	
=과 ==의 차이는?	
print()와 return의 차이는?	
클래스와 객체의 관계는?	

2. 자료형과 변수

① 자료형을 비교하며 정리하기

자료형	예시	핵심 특징
정수 int	25	정수 계산
실수 float	3.5	소수점이 있는 수
문자열 str	"Python"	순서 있음, 문자 직접 변경 불가
리스트 list	[70, 80, 90]	순서 있음, 원소 변경 가능
튜플 tuple	(70, 80, 90)	순서 있음, 원소 대입·삭제 불가
딕셔너리 dict	{"name": "민지", "score": 90}	키로 값 조회
집합 set	{1, 2, 3}	중복 제거, 인덱싱 불가
불 bool	True, False	참·거짓 표현

연산자 확인

print(9 / 2)	
print(9 // 2)	
print(9 % 2)	
print(2 ** 4)	

- //는 결과를 아래쪽 정수로 내립니다. 따라서 $-9 // 2$ 는 -5 입니다.
- 원소가 하나인 튜플은 (10,)처럼 쉼표가 필요합니다.
- 빈 딕셔너리는 {}, 빈 집합은 set()입니다.
- 0, "", [], {}, None등은 조건식에서 거짓으로 판단됩니다.

② 문자열: 인덱싱과 슬라이싱

해싱은 인덱스는 0부터, 슬라이싱의 끝 위치는 포함하지 않는다는 점입니다. 문자열 메서드로 만든 결과를 보관하려면 변수에 대입해야 합니다.

```
text = "Python"

print(text[0])      # P
print(text[-1])     # n
print(text[1:4])    # yth
print(text[:3])     # Pyt
print(text[::2])    # Pto

message = "  hello python  "
print(message.strip().upper()) # HELLO PYTHON

subjects = "국어,영어,수학".split(",")
print(subjects)      # ['국어', '영어', '수학']
print("/".join(subjects)) # 국어/영어/수학
```

③ 리스트: 변경과 메서드 반환값

append()와 sort()는 리스트 자체를 변경합니다. 특히 sort()의 반환값과 정렬된 리스트를 혼동하지 않도록 합니다.

```
scores = [75, 90, 60]

scores.append(85)
scores[0] = 80
scores.sort()

print(scores)      # [60, 80, 85, 90]
print(scores[-1])  # 90
print(scores[1:3]) # [80, 85]
```

다음 코드는 오류를 발생시키지는 않지만, 기대한 결과와 다릅니다.

```
numbers = [3, 1, 2]
result = numbers.sort()

print(numbers) # [1, 2, 3]
print(result)  # None
```

“원본을 정렬한 것과 정렬 결과를 반환받는 것은 구분해야 합니다.”

④ 변수: 같은 리스트를 가리키는 경우

```
a = [10, 20]
b = a
b[0] = 99

print(a) # [99, 20]
print(b) # [99, 20]
```

b = a는 리스트를 새로 복사하지 않습니다. 두 변수가 같은 리스트를 가리킵니다.

```
a = [10, 20]
b = a.copy()
b[0] = 99

print(a) # [10, 20]
print(b) # [99, 20]
```

이 예제처럼 숫자만 들어 있는 리스트에서는 copy()로 별도 리스트를 만들어 사용할 수 있습니다. 중첩 리스트는 내부 객체가 공유될 수 있습니다.

⑤ 실습해 보기

다음 자료를 사용합니다.

```
student = {"name": "민지", "scores": [80, 70, 90]}
```

1. 이름 출력하기
2. 점수 리스트에 100 추가하기
3. 평균 구하기
4. 이름과 평균을 한 문장으로 출력하기

정답코드

3. 조건문과 반복문

① 조건문: 조건의 순서 확인하기

if와 elif는 위에서부터 검사하며, 참인 분기를 만나면 해당 분기만 실행합니다.

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "D"

print(grade) # B
```

오류 찾기

```
score = 95

if score >= 60:
    print("D")
elif score >= 90:
    print("A")
```

확인할 표현:

```
score >= 60 and attendance >= 80 # 두 조건을 모두 만족
score >= 90 or bonus == True     # 둘 중 하나 이상 만족
name in students                 # 포함 여부
```

② 반복문: 반복 대상과 종료 조건

문법	사용 상황	주의점
for	자료의 원소를 차례로 처리	반복 대상 확인
while	조건이 참인 동안 반복	조건이 거짓이 되도록 상태 갱신
break	가장 가까운 반복문 종료	전체 프로그램 종료가 아님
continue	현재 회차의 나머지를 건너뛸	다음 반복으로 이동
range(1, 5)	1부터 4까지 순회	끝값 5는 제외

실행 결과 예상하기

```
total = 0

for number in range(1, 7):
    if number % 2 == 0:
        continue
    total += number

print(total) # 9
```

- 같은 결과를 구하는 리스트 컴프리헨션

```
odds = [n for n in range(1, 7) if n % 2 != 0]
print(odds)      # [1, 3, 5]
print(sum(odds)) # 9
```

③ 실습: 합격자 수 구하기

조건: 다음 점수 중 70점 이상인 점수만 출력하고 합격자 수를 구합니다.

scores = [55, 80, 70, 95, 60]

출력:

80

70

95

합격자: 3명

- while문의 종료 조건을 질문합니다.

```
count = 3

while count > 0:
    print(count)
    count -= 1
```

“count -= 1을 지우면 왜 반복이 끝나지 않을까요?”

4. 함수

① 함수의 핵심 구조

함수는 처리할 내용을 묶고, 인수를 받아 계산한 값을 반환할 수 있습니다. return을 만나면 함수 실행이 종료되고, 명시적인 반환값 없이 끝나면 None을 반환합니다.

```
def get_average(scores):    # scores: 매개변수
    return sum(scores) / len(scores)

result = get_average([80, 90, 100]) # 리스트: 인수
print(result)                      # 90.0
```

반드시 구분할 내용

구분	의미
매개변수	함수 정의에서 값을 받는 변수
인수	함수 호출 때 전달하는 값
print()	화면에 출력
return	호출한 곳으로 값을 돌려주고 함수 종료
지역 변수	함수 내부에서 사용되는 변수
기본값	인수를 생략했을 때 사용할 값

② print()와 return비교

```
def show_double(number):
    print(number * 2)

def get_double(number):
    return number * 2

a = show_double(5)
b = get_double(5)

print("a:", a)
print("b:", b)
```

출력:

10

a: None

b: 10

질문

“화면에 10이 보였는데, 왜 a에는 10이 저장되지 않았을까요?”

③ 기본값과 변수의 범위

```
def is_pass(score, standard=60):  
    return score >= standard  
  
print(is_pass(65))      # True  
print(is_pass(65, 70)) # False
```

```
score = 80  
  
def change_score():  
    score = 100  
    return score  
  
result = change_score()  
  
print(score) # 80  
print(result) # 100
```

함수 안의 score와 바깥의 score를 구분!!

④ 나머지 함수 문법

문법	기억할 내용
def total(*args):	여러 위치 인수를 튜플로 받음
def profile(**kwargs):	여러 키워드 인수를 딕셔너리로 받음
return a, b	두 값을 담은 튜플 하나를 반환
global count	함수 안에서 전역 이름에 대입할 때 사용
lambda x: x * 2	간단한 함수를 표현
함수 첫 줄의 문자열	독스트링으로 함수 설명 작성

⑤ 실습: 평균으로 합격 판단하기

조건:

- 점수 리스트를 전달받습니다.
- 평균이 70점 이상이면 "합격", 아니면 "불합격"을 반환합니다.
- 점수 리스트에는 하나 이상의 점수가 있다고 가정합니다.

정답

5. 사용자·파일·프로그램 입출력

① 사용자 입력: 자료형 변환

input()으로 입력받은 값은 문자열입니다. 숫자 계산에는 변환이 필요합니다.

```
name = input("이름: ")  
score = int(input("점수: "))  
  
print(f"{name}님의 점수는 {score}점입니다.")
```

다음 두 결과를 비교합니다.

```
print("80" + "10")          # 8010
print(int("80") + int("10")) # 90
```

② 파일 읽고 쓰기

모드·메서드	의미
"r"	읽기
"w"	쓰기, 기존 내용 덮어쓰기
"a"	파일 끝에 추가
read()	남은 내용 전체 읽기
readline()	한 줄 읽기
readlines()	줄들을 리스트로 읽기

with문을 사용하면 블록을 벗어날 때 파일이 자동으로 닫힙니다.

```
with open("review_result.txt", "w", encoding="utf-8") as file:
    file.write("민지,85\n")
    file.write("준호,90\n")

with open("review_result.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

출력:

민지,85

준호,90

③ 프로그램의 입출력: 실행할 때 인수 전달하기

4장의 sys.argv는 프로그램을 실행할 때 전달한 인수를 읽는 내용입니다.

```
# welcome.py
import sys

print(sys.argv[0])
print(sys.argv[1])
```

터미널에서 실행:

python welcome.py 민지

출력:

welcome.py

민지

- sys.argv[0]: 실행한 스크립트 이름
- sys.argv[1]: 첫 번째 인수
- 인수를 생략하면 이 코드의 sys.argv[1] 접근에서 오류가 발생합니다.

6. 클래스

① 핵심 용어와 구조

용어	의미	예시
클래스	객체의 데이터와 동작을 정의	Student
객체·인스턴스	클래스로 생성한 개별 대상	s1, s2
인스턴스 변수	각 객체에 저장되는 속성	self.name
메서드	클래스 안에 정의한 함수	result()
self	메서드를 호출한 객체를 받는 매개변수	s1.result()의 s1
__init__()	객체 생성 과정에서 초기 상태를 설정	이름·점수 저장

```
class Student:
    school = "미래대학교"

    def __init__(self, name, score):
        self.name = name
        self.score = score

    def result(self):
        if self.score >= 70:
            return "합격"
        return "불합격"

s1 = Student("민지", 85)
s2 = Student("준호", 65)

print(s1.name, s1.result())
print(s2.name, s2.result())
```

출력:

민지 합격

준호 불합격

② self를 실제 객체와 연결하기

s1.result()

Student.result(s1)

“s1이 호출하면 self는 s1을 가리키고, s2가 호출하면 self는 s2를 가리킵니다.”

self.name = name

- 오른쪽 name: 호출할 때 전달받은 값
- 왼쪽 self.name: 객체에 저장할 속성

③ 객체별 값과 클래스 변수 비교하기

앞의 코드에 이어 실행

```
s1.score = 95

print(s1.score) # 95
print(s2.score) # 65

Student.school = "한국대학교"

print(s1.school) # 한국대학교
print(s2.school) # 한국대학교
```

점수는 객체별로 저장되고, 학교명은 클래스 속성에서 조회됩니다.

단, s1.school = "다른대학교"처럼 객체에 직접 대입하면 s1의 별도 속성이 생겨 클래스 속성을 가립니다.

④ 상속과 오버라이딩

앞의 Student클래스를 그대로 두고 다음 코드를 추가합니다.

```
class ScholarshipStudent(Student):
    def result(self):
        if self.score >= 90:
            return "장학 대상"
        return "장학 제외"

s3 = ScholarshipStudent("서연", 92)

print(s3.name)      # 서연
print(s3.result())  # 장학 대상
```

- **상속**: Student의 기능을 이어받습니다. 이 예제는 부모의 `__init__()`를 사용합니다.
- **오버라이딩**: 부모의 `result()`를 자식 클래스에서 다시 정의합니다.

⑤ 질문

1. s1.score를 바꾸면 s2.score도 바뀌나요?
2. s1.result()를 호출할 때 self를 직접 전달하지 않는 이유는?
3. s3.result()는 부모와 자식 중 어느 메서드를 실행하나요?

정답:

7. 마무리

번호	문제
1	<code>9 // 2</code> , <code>9 % 2</code> 의 결과를 각각 쓰세요.
2	<code>"Python"[2:5]</code> 의 결과는?
3	<code>a = [1, 2]</code> , <code>b = a</code> , <code>b.append(3)</code> 실행 후 <code>a</code> 는?
4	원소가 하나인 튜플을 작성하세요. 값은 10입니다.
5	<code>list(range(2, 8, 2))</code> 의 결과는?
6	<code>break</code> 와 <code>continue</code> 의 차이를 설명하세요.
7	명시적인 반환값 없이 끝난 함수가 반환하는 값은?
8	<code>input()</code> 의 반환 자료형은?
9	기존 내용을 유지하면서 파일 끝에 추가하는 모드는?
10	인스턴스 메서드의 <code>self</code> 는 무엇을 가리키나요?